

torch.nn.functional.conv_transpose3d#

https://pytorch.org/docs/stable/generated/torch.nn.functional.conv_transpose3d.html

Description:

Applies a 3D transposed convolution operator over an input image composed of several input planes, sometimes also called “deconvolution”. This operator supports `TensorFloat32`. See `ConvTranspose3d` for details and output shape. In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See [Reproducibility](#) for more information.

Function Signature

```
torch.nn.functional.conv_transpose3d(input, weight,
bias=None, stride=1, padding=0, output_padding=0, groups=1,
dilation=1) → Tensor#
```

Parameters

Code Examples

```
>>> inputs = torch.randn(20, 16, 50, 10, 20)
>>> weights = torch.randn(16, 33, 3, 3, 3)
>>> F.conv_transpose3d(inputs, weights)
```

Notes & Warnings

NOTE In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See [Reproducibility](#) for more information.

Detailed Documentation

Documentation

Applies a 3D transposed convolution operator over an input image composed of several input planes, sometimes also called “deconvolution”

This operator supports `TensorFloat32`.

See `ConvTranspose3d` for details and output shape.

In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See [Reproducibility](#) for more information.

`input` – input tensor of shape $(\text{minibatch}, \text{in_channels}, i\text{T}, i\text{H}, i\text{W})$ $(\text{minibatch}, \text{in_channels}, i\text{T}, i\text{H}, i\text{W})$

`weight` – filters of shape $(\text{in_channels}, \text{out_channelsgroups}, k\text{T}, k\text{H}, k\text{W})$ $(\text{in_channels}, \frac{\text{out_channels}}{\text{groups}}, k\text{T}, k\text{H}, k\text{W})$ $(\text{in_channels}, \text{groupsout_channels}, k\text{T}, k\text{H}, k\text{W})$

`bias` – optional bias of shape (out_channels) (out_channels) .
Default: None

`stride` – the stride of the convolving kernel. Can be a single number or a tuple $(s\text{T}, s\text{H}, s\text{W})$. Default: 1

`padding` – $\text{dilation} * (\text{kernel_size} - 1)$ - padding zero-padding will be added to both sides of each dimension in the input. Can be a single number or a tuple $(\text{padT}, \text{padH}, \text{padW})$.
Default: 0

`output_padding` – additional size added to one side of each dimension in the output shape. Can be a single number or a tuple $(\text{out_padT}, \text{out_padH}, \text{out_padW})$. Default: 0

`groups` – split input into groups, in_channels in_channels should be divisible by the number of groups. Default: 1

`dilation` – the spacing between kernel elements. Can be a single number or a tuple $(d\text{T},$

dH, dW). Default: 1